



โครงการ

Battle Station

จัดทำโดย

6704062612367 นายณัฐชนน พรินทร์

เสนอ

ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

วิชา 040613204 Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2568

ภาควิชาวิทยาศาสตร์คอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

บทที่ 1

บทนำ

เกี่ยวกับโปรเจค

ชื่อ: Battle Control

นำเสนอโดย: นายณัฐชนน พรินทร์

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

Source code: [Github](#)

ที่มาและความสำคัญ

โครงการนี้จัดขึ้นเพื่อวัดผลความสามารถในการเรียนวิชา Object Oriented Programming โดยการนำเรื่องที่เรียนมาสร้างเป็นชิ้นงานในรูปแบบเกมโดยใช้แนวคิดการเขียนโปรแกรมแบบเชิงวัตถุ และยังช่วยให้ผู้จัดทำเรียนรู้อุปกรณ์และเครื่องมือ ผู้จัดทำได้สร้างเกมนี้ขึ้นมา

ประเภทของโครงการ

เกมคอมพิวเตอร์ (Desktop Java application)

ประโยชน์

- เพื่อความสนุกสนาน
- ฝึกความคิดเชิงตรรกะ
- ฝึกความว่องไวในการคิด
- มีแนวทางการรับมือเมื่อมีข้าศึกมารุกรานประเทศ เพื่อป้องกันการยึดอำนาจของฝ่ายตรงข้าม

ขอบเขตของโครงการ

รายการ	1-7 ก.ย.	8-14 ก.ย.	15-21 ก.ย.	22-30 ก.ย.	1-7 ต.ค.
Graphic and Art Design					
Learn how to actually do this					
Programming					
Documentation					
Debug					

บทที่ 2

ส่วนการพัฒนา

รายละเอียดเกม

เกมประเภท RTS ผสม Tower Defense สร้างฐานและแนวป้องกันอย่าให้ผู้บุกรุกเข้ามา โจมตีฐานของเราได้ต่อกรกับศัตรูมากมายที่รู้จักแต่การทำลายล้างและต้องการที่จะช่วงชิง อิสรภาพของประชาชนของเราไป

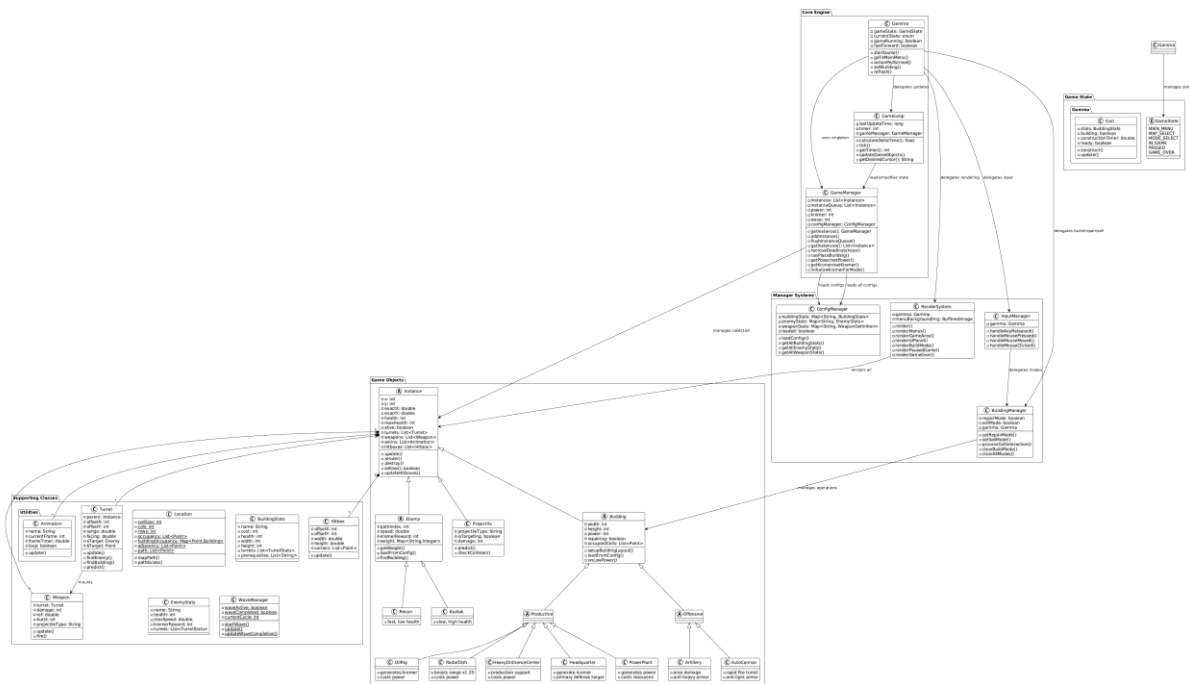
เนื้อเรื่องย่อ

ในโลกที่อำนาจคือทุกสิ่ง การแย่งชิงอำนาจและผลประโยชน์กลายเป็นสิ่งที่ต้องเกิดขึ้นอย่างหลีกเลี่ยงไม่ได้ กองทัพแดง (The Red Army) ได้ฉีกสนธิสัญญาสันติภาพแห่งโลกและเริ่มต้นการบุกรุกที่คาดไม่ถึงถึงหลายประเทศถูกโจมตีอย่างกะทันหันทำให้ถูกยึดครองและขยายอำนาจของกองทัพแดง ประเทศที่เหลืออยู่รู้ดีว่าไม่สามารถต่อต้านการคุกคามระดับโลกนี้ด้วยตัวเอง จึงรวมตัวกันก่อตั้งมหาอำนาจใหม่ชื่อว่า สหพันธมิตร (The Allied Nations) แต่การรวมตัวครั้งใหญ่นี้มันก็ยังไม่ใช่เพียงพอต่อการสู้อำนาจจากกองทัพแดง สหพันธมิตรยังคงสูญเสียฐานที่มั่นและจุดที่สำคัญทางยุทธศาสตร์หลายแห่งไป จนกระทั่งพวกเขาได้ค้นพบวิธีควบคุมเวลาจากการระดมสมองของหัวกะทิแต่ละชาติ ถึงจะยังไม่เสถียรมากนัก เกมก็อาจจะเปลี่ยนเล็กน้อย

วิธีการเล่น

- ฐานทัพใหญ่ (Headquarter) และเส้นทางที่ศัตรูจะเคลื่อนที่จะถูกกำหนดไว้ เราต้องสร้างฐานจากแผ่นควบคุมทางขวาเพื่อป้องกันไม่ให้ศัตรูเข้ามาทำลายฐานทัพใหญ่ ส่วนใหญ่แล้วสิ่งก่อสร้างมักจะต้องมีสิ่งอื่นสร้างไว้อยู่แล้วถึงจะเริ่มก่อสร้างได้ (Prerequisites) และต้องใช้ทรัพยากรกับเวลาในการสร้าง
- ไม่สามารถสร้าง 2 สิ่งจากหมวดเดียวกันพร้อมกันได้
- หากยังไม่เริ่มเกม เวลาที่ต้องใช้สร้างจะลดลงอย่างมาก
- หากพลังงานไม่เพียงพอ ($\text{Power} < 0$) ยูนิโปรแกรมจะมีประสิทธิภาพน้อยลง
- กด Z เพื่อเข้าสู่โหมดซ่อม (Repair Mode)
- กด X เพื่อเข้าสู่โหมดขาย (Sell Mode)

Class diagram



Class Diagram นี้แบ่งออกเป็น 5 ชั้นหลัก ดังนี้

1. Core Engine Layer (ชั้นหลักของเกม)

- **Gamma:** หัวใจของโปรเจกต์ ควบคุมการเปลี่ยนหน้าจอทั้งหมด
 - เป็นแพลตฟอร์มหลักในการเล่นเกมน ควบคุมการเปลี่ยนหน้าจอทั้งหมด
 - สลับระหว่าง MAIN_MENU → MAP_SELECT → MODE_SELECT → IN_GAME/PAUSED
 - จัดการคำสั่งเกม
- **GameManager:** จัดการสิ่งที่อยู่ในเกม
 - จัดการ instance ทั้งหมดในเกม (อาคาร, ศัตรู, กระสุน)
 - บัฟเฟอร์ instance ใหม่ด้วย instanceQueue เพื่อป้องกัน concurrent modification
 - จัดการทรัพยากร (power, kromer)
 - ตรวจสอบการวางอาคาร
 - เก็บ ConfigManager
- **GameLoop:** รัน 60 รอบต่อวิ
 - คำนวณ delta time สำหรับการอัปเดตที่สม่ำเสมอ
 - อัปเดต universal timer
 - เรียก update() สำหรับ instance ทั้งหมด

- จัดการระบบ wave
- ตรวจสอบ RadarDish เพื่อเพิ่ม range boost (x1.25)

2. Manager Systems Layer (ระบบจัดการ)

- **RenderSystem:** วาดภาพ
 - วาดทุกอย่างบนหน้าจอ: เมนู, พื้นที่เกม, UI panel, โหมดสร้าง
 - จัดการการแสดงผลตามสถานะเกม (pause, game over)
 - แสดง build preview (ช่องที่วางได้/วางไม่ได้, ระยะของป้อม)
- **InputManager:** ควบคุม Input
 - จัดการ keyboard/mouse events
 - สลับโหมด (repair mode: Z, sell mode: X, pause: ESC)
 - ส่งต่อ interaction ไปยัง BuildingManager
- **BuildingManager:** ควบคุมการประมวลผลสิ่งก่อสร้าง
 - จัดการ build/repair/sell mode (โหมดต่างๆ ไม่ทับซ้อนกัน)
 - ประมวลผลการคลิกบนตาราง
 - จัดการข้อมูล build preview (ตำแหน่งป้อม, ระยะ, ช่องที่วางได้)
- **ConfigManager:** ดึงค่าจาก json
 - โหลดไฟล์ JSON configuration (buildings.json, enemies.json, weapons.json)
 - แคช stats ทั้งหมดใน HashMap
 - โหลดครั้งเดียวตอน startup (lazy loading)

3. Game Object Layer (วัตถุในเกม)

- **Instance:** คลาสพื้นฐานของวัตถุทั้งหมด
 - มี position (x, y, exactX, exactY)
 - มี health และสถานะ alive
 - มี turrets, weapons, animations, hitboxes
 - กำหนด method abstract: update(), render(), destroy()
- **Building:** สิ่งก่อสร้าง
 - มีขนาด (width, height)
 - ใช้/ผลิต power

- ครอบครองช่องในตาราง (occupiedCells)
- สามารถซ่อมแซมได้ (repairing)
- **Enemy:** ศัตรู
- เดินตามเส้นทาง (pathIndex)
- มีความเร็ว (speed)
- ให้รางวัล kromer เมื่อถูกทำลาย
- มี weight สำหรับเลือกเป้าหมาย
- **Projectile:** กระสุน
- มีประเภท (projectileType)
- ติดตามเป้าหมาย (isTargeting)
- ตรวจสอบการชน (checkCollision)
- คำนวณการเคลื่อนที่ล่วงหน้า (predict)

4. Supporting Classes Layer (คลาสสนับสนุน)

- **Turret:** ป้อมปืน
 - ติดกับ parent instance
 - มี offset (offsetX, offsetY)
 - มีระยะยิง (range)
 - หมุนหาเป้าหมาย (facing)
 - มีเป้าหมายเป็น Enemy หรือ Building
- **Weapon:** อาวุธของป้อม
 - ติดกับ Turret
 - มีความเสียหาย (damage)
 - มี rate of fire (rof)
 - มี burst count
 - ยิงกระสุนประเภทต่างๆ
- **Hitbox:** กล่องชนสำหรับตรวจสอบการโดน
 - มี offset และขนาด
 - มีมุมทั้ง 4 (corners)
 - อัปเดตตามตำแหน่ง instance
- **Utilities.Animation:** ระบบแอนิเมชัน

- เล่นจากไฟล์ภาพต่อเนื่อง (frame001.png, frame002.png, ...)
- ควบคุมด้วย frameTimer (60 FPS)
- วนซ้ำได้ (loop)
- **Location:** ข้อมูลแผนที่และตาราง
 - กำหนดขนาดช่อง (cellSize), จำนวนแถว/คอลัมน์ (rows, cols)
 - ติดตามช่องที่ถูกครอบครอง (occupancy, buildingOccupancy)
 - เก็บเส้นทางของศัตรู (path)
 - ตรวจสอบความเชื่อมต่อของเส้นทาง
- **WaveManager:** ควบคุม Wave
 - ตรวจสอบสถานะ wave (active, completed)
 - นับรอบ (currentCycle)
 - เริ่ม wave และอัปเดตความสมบูรณ์

5. Game State Layer (สถานะเกม)

- **GameState (enum):** สถานะหน้าจอ
 - MAIN_MENU: หน้าเมนูหลัก
 - MAP_SELECT: เลือกแผนที่
 - MODE_SELECT: เลือกโหมดเกม (normal, survival, challenge, sandbox)
 - IN_GAME: กำลังเล่นเกม
 - PAUSED: หยุดชั่วคราว
 - GAME_OVER: จบเกม
- **Gamma.Icon:** ไอคอนอาคารใน UI
 - เก็บ BuildingStats
 - ตรวจสอบสถานะการสร้าง (building, ready)
 - มี construction timer

รูปแบบการพัฒนา

- ภาษา: Java
- IDE: [Visual Studio Code](#)
- การพัฒนา GUI: javax.swing (JFrame, JPanel, Graphics2D)
- การจัดการรูปภาพ: java.awt.image.BufferedImage

- เครื่องมือที่ใช้ทำ asset: [Blender](#)

แนวคิดการเขียนโปรแกรมเชิงวัตถุ

Constructor

```
Gamma - Gamma.java

174 public Gamma() {
175     gameInstance = this; // Set static reference
176     if (gameManager == null)
177         gameManager = new GameManager(); // Initialize once
178     if (gameLoop == null)
179         gameLoop = new GameLoop(gameManager); // Initialize once
180     inputManager = new InputManager(this); // REFACTORED: Initialize input manager
181     renderSystem = new RenderSystem(this); // REFACTORED: Initialize render system
182     buildingManager = new BuildingManager(this); // REFACTORED: Initialize building manager
183
184     // Load all configuration from JSON files
185     gameManager.getConfigManager().loadConfigs();
186
187     setPreferredSize(new Dimension(1920, 1080));
188     setBackground(Color.BLACK);
189     setDoubleBuffered(true); // Important for smooth rendering
190     setFocusable(true);
191     requestFocusInWindow();
192
193     // let it all begins
194     instances = gameManager.getInstances();
195     power = gameManager.getPower();
196     kromer = gameManager.getKromer();
197     gameRunning = false;
198     fastForward = false;
199     lastUpdateTime = System.currentTimeMillis();
200
201     // create a timer that'll update 60 times per second (60 FPS)
202     gameTimer = new Timer(FRAME_TIME, this);
203
204     addKeyListener(this);
205     addMouseListener(this);
206     addMouseMotionListener(this);
207     init();
208     Utilities.setCustomCursor(this, "normal_cursor", -16, -16);
209     // Start timer immediately so menus render even when the game isn't running
210     gameTimer.start();
211 }
```

Constructor ของ Gamma นี้ทำหน้าที่ในการกำหนดค่าเริ่มต้นของเกมทั้งหมด เช่น ขนาดหน้าต่าง สี และ สร้าง Manager ต่างๆ โดยตั้งค่าขนาดหน้าจอเป็น 1920x1080 พิกเซล กำหนดสีพื้นหลังเป็นสีดำ เปิดใช้งาน double buffering สร้าง Manager ต่างๆ (GameManager, GameLoop, InputManager, RenderSystem, BuildingManager) และสร้าง Timer ที่จะอัปเดตเกม 60 ครั้งต่อวินาที

Gamma - GameManager.java

```
20 public GameManager() {
21     this.instances = new ArrayList<>();
22     this.instanceQueue = new ArrayList<>();
23     this.configManager = new ConfigManager();
24
25     this.power = 0;
26     this.kromer = 1000;
27     this.wave = 0;
28
29     // Set as singleton instance
30     if (instance == null) {
31         instance = this;
32     }
33 }
```

Constructor นี้ทำหน้าที่ในการกำหนดค่าเริ่มต้นของ state เกม เช่น พลังงาน (power = 0) เงิน (kromer = 1000) และ wave (wave = 0) พร้อมกับตั้งค่า instance เป็น singleton

Encapsulation

Gamma - BuildingManager.java

```
21 private boolean repairMode = false;
22 private boolean sellMode = false;
23
24 // Reference to Gamma for state access
25 private Gamma gamma;
```

Gamma - BuildingManager.java

```
53 // ===== REPAIR MODE =====
54
55 public void setRepairMode(boolean enabled) {
56     if (enabled && sellMode) {
57         sellMode = false; // Can't be both
58     }
59     if (enabled && gamma.buildMode) {
60         clearBuildMode(); // Exit build mode when entering repair
61     }
62     repairMode = enabled;
63 }
64
65 public boolean isRepairMode() {
66     return repairMode;
67 }
68
69 public void toggleRepairMode() {
70     setRepairMode(!repairMode);
71 }
72
73 // ===== SELL MODE =====
74
75 public void setSellMode(boolean enabled) {
76     if (enabled && repairMode) {
77         repairMode = false; // Can't be both
78     }
79     if (enabled && gamma.buildMode) {
80         clearBuildMode(); // Exit build mode when entering sell
81     }
82     sellMode = enabled;
83 }
84
85 public boolean isSellMode() {
86     return sellMode;
87 }
88
89 public void toggleSellMode() {
90     setSellMode(!sellMode);
91 }
```

BuildingManager ใช้ Encapsulation โดยซ่อนตัวแปร repairMode และ sellMode เป็น private และให้เข้าถึงผ่าน public methods เท่านั้น (setRepairMode(), isRepairMode(), toggleRepairMode()) ทำให้สามารถควบคุมการเปลี่ยนแปลงค่าได้อย่างปลอดภัย และป้องกันไม่ให้ repairMode และ sellMode เปิดพร้อมกันได้

Gamma - GameManager.java

```
12 private List<Instance> instances;
13 private List<Instance> instanceQueue; // Buffer for new instances added during updates
14 private ConfigManager configManager;
15
16 private int power;
17 private int kromer;
18 private int wave;
```

Gamma - GameManager.java

```
97 // ===== RESOURCES =====
98
99 public int getPower() {
100     return power;
101 }
102
103 public void setPower(int power) {
104     this.power = power;
105 }
106
107 public void addPower(int amount) {
108     this.power += amount;
109 }
110
111 public int getKromer() {
112     return kromer;
113 }
114
115 public void setKromer(int kromer) {
116     this.kromer = kromer;
117 }
118
119 public void addKromer(int amount) {
120     this.kromer += amount;
121 }
```

GameManager ซ่อนข้อมูล (instances, power, kromer) เป็น private และให้เข้าถึงผ่าน getter/setter methods (getPower(), setPower(), addPower(), getKromer(), setKromer(), addKromer()) ทำให้สามารถควบคุมการเข้าถึงและแก้ไขข้อมูลได้อย่างปลอดภัย

Composition

Gamma - Gamma.java

```
37 private static GameManager gameManager; // Central game state
38 private static GameLoop gameLoop; // Timing and update logic
39 private InputManager inputManager; // Input handling
40 private RenderSystem renderSystem; // Rendering logic
41 private BuildingManager buildingManager; // Build/repair/sell logic
```

Gamma - Gamma.java

```
176 if (gameManager == null)
177     gameManager = new GameManager(); // Initialize once
178 if (gameLoop == null)
179     gameLoop = new GameLoop(gameManager); // Initialize once
180 inputManager = new InputManager(this); // Initialize input manager
181 renderSystem = new RenderSystem(this); // Initialize render system
182 buildingManager = new BuildingManager(this); // Initialize building manager
```

Gamma ใช้ Composition โดยประกอบด้วย Manager หลายตัว (GameManager, GameLoop, InputManager, RenderSystem, BuildingManager) แต่ละตัวมีหน้าที่เฉพาะ:

- GameManager: จัดการ state ของเกม
- GameLoop: จัดการเวลาและการอัปเดต
- InputManager: จัดการ input จากผู้เล่น
- RenderSystem: จัดการการแสดงผล
- BuildingManager: จัดการการสร้าง/ซ่อม/ขายอาคาร

```

Gamma - Gamma.java

843 ArrayList<Turret> turrets = new ArrayList<>();
844 ArrayList<Weapon> weapons = new ArrayList<>();
845 ArrayList<Utilities.Animation> anims = new ArrayList<>();
846 ArrayList<Hitbox> hitboxes = new ArrayList<>(); // Multiple hitboxes per instance

```

Instance ใช้ Composition โดยประกอบด้วย Turret (ป้อมปืน), Weapon (อาวุธ), Hitbox (กล่องชน) และ Animation (แอนิเมชั่น) ทำให้ Instance มีความสามารถที่ซับซ้อนโดยรวมส่วนประกอบย่อยเข้าด้วยกัน

Polymorphism

```

Gamma - Gamma.java

659 class Elements {
660     interface Renderable {
661         void render(Graphics2D g);
662     }
663
664     Renderable r;
665     int zIndex;
666
667     public Elements(Renderable r, int zIndex) {
668         this.r = r;
669         this.zIndex = zIndex;
670     }
671 }

```

```

Gamma - Gamma.java

836 abstract class Instance implements Elements.Renderable {

```

Instance implement interface Renderable ทำให้ทุก Instance (Building, Enemy, Projectile) ต้องมี method render() แต่แต่ละคลาสสามารถ implement การแสดงผลต่างกันได้

การ Override method update()

Gamma - Gamma.java

```
870 abstract void update(float deltaTime);
```

ทุก Instance ต้อง implement method update() แต่แต่ละคลาสสามารถมีพฤติกรรมการอัปเดตที่ต่าง
กัน:

- Building: อัปเดตบ่อนปืนและอาวุธ
- Enemy: อัปเดตการเคลื่อนที่ตามเส้นทาง
- Projectile: อัปเดตการเคลื่อนที่และตรวจสอบการชน

การใช้ Polymorphism ในการเก็บ Instance

Gamma - Gamma.java

```
42 private static List<Instance> Instances;
```

List Instances เก็บ Instance หลายประเภท (Building, Enemy, Projectile) และเรียก method
update() ได้เหมือนกันทั้งหมด ทั้งๆ ที่แต่ละคลาสมี implementation ที่แตกต่างกัน

Abstract Class

```
Gamma - Gamma.java

718 abstract class Location {
719     public static final int cellSize = 40; // size of each grid cell in pixels
720     public static final int cols = 39; // number of columns
721     public static final int rows = 27; // number of rows
722     public static ArrayList<Point> occupancy = new ArrayList<>(); // occupied cells for path and obstacles
723     public static HashMap<Point, Building> buildingOccupancy = new HashMap<>(); // occupied cells for buildings
724     public static ArrayList<Point> adjacency = new ArrayList<>(); // list of cells that a building provide adjacency
725                                     // for, or in other word, buildable cells
726     public static ArrayList<Point> path = new ArrayList<>(); // path for enemies to follow
727
728     public static Color bg; // background color
729     public static Color pc; // path color
730
731     public static Point start; // starting point for enemies
732
733     public Location() {
734     }
735
736     void mapPath(int x1, int y1, int x2, int y2) {
737         // Only create path if line is horizontal or vertical
738         if (x1 != x2 && y1 != y2) {
739             return; // Not horizontal or vertical, do nothing
740         }
741
742         // add points to path
743         if (x1 == x2) {
744             // Vertical line
745             if (y1 > y2) {
746                 for (int y = y1; y >= y2; y--) {
747                     if (!pathExists(x1, y)) {
748                         path.add(new Point(x1, y));
749                         occupancy.add(new Point(x1, y));
750                     }
751                 }
752             } else {
753                 for (int y = y1; y <= y2; y++) {
754                     if (!pathExists(x1, y)) {
755                         path.add(new Point(x1, y));
756                         occupancy.add(new Point(x1, y));
757                     }
758                 }
759             }
760         } else {
761             // Horizontal line
762             if (x1 > x2) {
763                 for (int x = x1; x >= x2; x--) {
764                     if (!pathExists(x, y1)) {
765                         path.add(new Point(x, y1));
766                         occupancy.add(new Point(x, y1));
767                     }
768                 }
769             } else {
770                 for (int x = x1; x <= x2; x++) {
771                     if (!pathExists(x, y1)) {
772                         path.add(new Point(x, y1));
773                         occupancy.add(new Point(x, y1));
774                     }
775                 }
776             }
777         }
778     }
779
780     boolean pathExists(int x, int y) {
781         for (Point p : path) {
782             if (p.x == x && p.y == y) {
783                 return true;
784             }
785         }
786         return false;
787     }
788 }
```

Location เป็น abstract class ที่กำหนดโครงสร้างพื้นฐานของแผนที่ (ขนาดช่อง จำนวนแถว/คอลัมน์ เส้นทาง) และมี method mapPath() สำหรับสร้างเส้นทาง

Gamma - Gamma.java

```
790 class Plain extends Location {
791     public Plain() {
792         bg = new Color(34, 139, 34);
793         pc = new Color(50, 175, 50);
794
795         start = new Point(43, 13);
796         mapPath(38, 13, 32, 13);
797         mapPath(32, 13, 32, 6);
798         mapPath(32, 6, 14, 6);
799         mapPath(14, 6, 14, 21);
800         mapPath(14, 21, 19, 21);
801         mapPath(19, 21, 19, 8);
802         mapPath(19, 8, 25, 8);
803         mapPath(25, 8, 25, 24);
804         mapPath(25, 24, 8, 24);
805         mapPath(8, 24, 8, 3);
806         mapPath(8, 3, 5, 3);
807
808         Gamma.add(new Empty());
809
810         Gamma.add(new Headquarter(1, 1));
811     }
812 }
```

Plain extends Location และสร้างแผนที่โดยกำหนดสีเขียว (bg = Color(34, 139, 34)) และสร้างเส้นทางเฉพาะของแผนที่

Gamma - Gamma.java

```
814 class OuterSpace extends Location {
815     public OuterSpace() {
816         bg = new Color(0, 0, 0);
817         pc = new Color(255, 255, 255);
818
819         start = new Point(-5, 25);
820         mapPath(0, 25, 37, 25);
821         mapPath(37, 25, 37, 1);
822         mapPath(37, 1, 1, 1);
823         mapPath(1, 1, 1, 20);
824         mapPath(1, 20, 34, 20);
825         mapPath(34, 20, 34, 4);
826         mapPath(34, 4, 11, 4);
827         mapPath(11, 4, 11, 11);
828         mapPath(11, 11, 19, 11);
829
830         Gamma.add(new Empty());
831
832         Gamma.add(new Headquarter(20, 10));
833     }
834 }
```

OuterSpace extends Location และสร้างแผนที่โดยกำหนดสีดำ (bg = Color(0, 0, 0)) และสร้างเส้นทางที่แตกต่างจาก Plain

```

836 abstract class Instance implements Elements.Renderable {
837     int x, y; // in cells
838     double exactX, exactY;
839     int health;
840     int maxHealth;
841     double facing = 0; // angle in radians
842     boolean alive = true;
843     ArrayList<Turret> turrets = new ArrayList<>();
844     ArrayList<Weapon> weapons = new ArrayList<>();
845     ArrayList<Utilities.Animation> anims = new ArrayList<>();
846     ArrayList<Hitbox> hitboxes = new ArrayList<>(); // Multiple hitboxes per instance
847     int zIndex = 3; // Default Z-index (higher than buildings, for buildings/enemies)
848     double timer = 0; // general purpose timer
849     double trueTimer = 0; // general purpose timer that isn't affected by time dilation
850     double healthMult = 1, damageMult = 1, speedMult = 1, rangeMult = 1, rofMult = 1;
851
852     BufferedImage image;
853     int imgX = Location.cellSize / 2, imgY = Location.cellSize / 2; // image offset
854     int scaleX = 1, scaleY = 1; // image scale
855
856     public Instance(int x, int y, int health) {
857         this.x = x;
858         this.y = y;
859         exactX = x * Location.cellSize + Location.cellSize / 2.0;
860         exactY = y * Location.cellSize + Location.cellSize / 2.0;
861         this.health = (int) (health * healthMult);
862         this.maxHealth = (int) (health * healthMult);
863     }
864
865     public Instance(double x, double y) { // for projectiles
866         this.exactX = x;
867         this.exactY = y;
868     }
869
870     abstract void update(float deltaTime);
871
872     public abstract void render(Graphics2D g);
873
874     void add(Turret turret) {
875         turrets.add(turret);
876     }
877
878     void add(Weapon weapon) {
879         weapons.add(weapon);
880     }
881
882     void add(Utilities.Animation animation) {
883         anims.add(animation);
884     }
885
886     // Add a hitbox to this instance
887     void hitbox(int offsetX, int offsetY, double width, double height) {
888         hitboxes.add(new Hitbox(width, height, offsetX, offsetY));
889         updateHitboxes();
890     }
891
892     void routine(float deltaTime) {
893         timer += deltaTime;
894         trueTimer += Gamma.FRAME_TIME / 1000;
895         if (this instanceof Building)
896             return;
897         updateHitboxes();
898     }
899
900     // Update all hitboxes (call this every frame for moving instances)
901     public void updateHitboxes() {
902         for (Hitbox hitbox : hitboxes) {
903             hitbox.update(exactX, exactY, facing);
904         }
905     }
906
907     // Check if instance is still alive (for cleanup)
908     public boolean isAlive() {
909         return alive;
910     }
911
912     // Destroy this instance
913     abstract public void destroy();
914
915     // Helper method to create weapons from config stats
916     protected Weapon createWeaponFromStats(Turret turret, WeaponStats stats) {
917         // Create a weapon with all stats (merged from weapons.json via name reference)
918         Weapon weapon = new Weapon(turret, stats.offsetX, stats.offsetY);
919         weapon.name = stats.name;
920         weapon.damage = stats.damage;
921         weapon.rof = stats.rof;
922         weapon.pSpeed = stats.projectileSpeed;
923         weapon.spread = stats.spread;
924         weapon.pierce = stats.pierce;
925         weapon.burst = stats.burst;
926         weapon.burstDelay = stats.burstDelay;
927         weapon.projectileType = stats.projectileType != null ? stats.projectileType : "bullet";
928         return weapon;
929     }
930 }

```

Instance เป็น abstract class ที่กำหนดคุณสมบัติพื้นฐานของวัตถุในเกม (ตำแหน่ง เลือด บ่อมปืน) และกำหนด abstract methods (update(), render(), destroy()) ที่คลาสย่อยต้อง implement

Gamma - Gamma.java

```
960 abstract class Building extends Instance {
```

Building เป็น abstract class ที่สืบทอดจาก Instance และเพิ่มคุณสมบัติเฉพาะของอาคาร (ขนาด width/height พลังงาน power การซ่อม repairing)

Inheritance

การสืบทอดของอาคารผลิตทรัพยากร (Productive)

abstract class Productive

Gamma - Gamma.java

```
1156 abstract class Productive extends Building { // declared only for type distinction
1157     public Productive(int x, int y, int health, int width, int height, int power) {
1158         super(x, y, health, width, height, power);
1159     }
1160 }
```

Productive เป็น abstract class ที่สืบทอดจาก Building สำหรับแยกประเภทอาคารผลิตทรัพยากร

คลาสย่อย Headquarter

Gamma - Gamma.java

```
1163 class Headquarter extends Productive {
1164     static final int width = 4;
1165     static final int height = 4;
1166
1167     public Headquarter(int x, int y) {
1168         super(x, y, 2000, width, height, 25);
1169         zIndex = 0; // Buildings on bottom layer
1170     }
1171 }
```

Headquarter extends Productive โดยกำหนด width = 4, height = 4, health = 2000, power = 25

การสืบทอดของอาคารโจมตี (Offensive)

abstract class Offensive

Gamma - Gamma.java

```
1215 abstract class Offensive extends Building { // declared only for type distinction
1216     public Offensive(int x, int y, int health, int width, int height, int power) {
1217         super(x, y, health, width, height, power);
1218     }
1219 }
```

Offensive เป็น abstract class ที่สืบทอดจาก Building สำหรับแยกประเภทอาคารโจมตี

คลาสย่อย AutoCannon

Gamma - Gamma.java

```
1222 class AutoCannon extends Offensive {
1223     static final int width = 1;
1224     static final int height = 1;
1225
1226     public AutoCannon(int x, int y) {
1227         super(x, y, 100, width, height, -10);
1228         zIndex = 0; // Buildings on bottom layer
1229     }
1230 }
```

AutoCannon extends Offensive โดยกำหนด width = 1, height = 1, health = 100, power = -10

การสืบทอดของศัตรู (Enemy)

abstract class Enemy

Gamma - Gamma.java

```
1245 abstract class Enemy extends Instance {
```

Enemy เป็น abstract class ที่สืบทอดจาก Instance และเพิ่มคุณสมบัติเฉพาะของศัตรู (ความเร็ว เส้นทาง รางวัล)

คลาสย่อย Recon และ Kodiak

Gamma - Gamma.java

```
1525 class Recon extends Enemy {  
1526     public Recon(int x, int y) {  
1527         super(x, y);  
1528     }  
1529 }  
1530  
1531 class Kodiak extends Enemy {  
1532     public Kodiak(int x, int y) {  
1533         super(x, y);  
1534     }  
1535 }
```

Recon และ Kodiak extends Enemy โดย Recon มีความเร็วสูงแต่เลือดน้อย ส่วน Kodiak มีความเร็วต่ำแต่เลือดมาก

ค่า stats ที่เห็นทั้งหมดนี้ จริง ๆ แล้วควรควบคุมด้วย JSON จาก class ของมันเอง แต่เนื่องจากเวลาไม่เพียงพอ จึงยังไม่สามารถย้ายระบบไปได้หมด

Headquarter ใน buildings.json

Gamma - buildings.json

```
2  {
3    "name": "Headquarter",
4    "buildingType": "productive",
5    "cost": 1000000,
6    "buildTime": 999999,
7    "buildRange": 6,
8    "width": 4,
9    "height": 4,
10   "health": 100,
11   "power": 25,
12   "description": "Don't let me down",
13   "imageName": "headquarter",
14   "buildLimit": 1,
15   "prerequisites": ["Peasant's tax money"]
16 },
```

Algorithm ที่สำคัญ

GameLoop คำนวณ deltaTime (เวลาที่ผ่านไปในการ 1 รอบ update) และ update ทุกอย่าง 60 รอบต่อวินาที

```

13 public class GameLoop {
14     private long lastUpdateTime = 0;
15     private int timer = 0; // Increments each update, used for animations
16
17     private final GameManager gameManager;
18
19     public GameLoop(GameManager gameManager) {
20         this.gameManager = gameManager;
21     }
22
23     /**
24      * Reset the game loop state (called when starting a new game)
25      */
26     public void reset() {
27         lastUpdateTime = 0;
28         timer = 0;
29     }
30
31     /**
32      * Calculate delta time based on current time and time scale.
33      * Also accounts for wave 0 which updates slower initially.
34      */
35     public float calculateDeltaTime(boolean fastForward, int wave) {
36         long currentTime = System.currentTimeMillis();
37         float timeScale = Math.min(fastForward ? 500.0f : 1000.0f, wave == 0 ? 10.0f : 1000.0f);
38         float deltaTime = lastUpdateTime == 0 ? 0 : (currentTime - lastUpdateTime) / timeScale;
39         lastUpdateTime = currentTime;
40         return deltaTime;
41     }
42
43     /**
44      * Increment the universal game timer (used for animations and timing)
45      */
46     public void tick() {
47         timer++;
48     }
49
50     /**
51      * Get the current universal game timer
52      */
53     public int getTimer() {
54         return timer;
55     }
56
57     /**
58      * Update all game objects and state.
59      * This is the main update loop.
60      */
61     public void updateGameObjects(float deltaTime,
62                                 List<Instance> instances, List<Instance> iQueue) {
63
64         // Update wave system
65         int currentWave = gameManager.getWave();
66         WaveManager.update(deltaTime, currentWave);
67         int newWave = WaveManager.updateWaveCompletion(deltaTime, currentWave);
68         if (newWave != currentWave) {
69             gameManager.setWave(newWave);
70         }
71
72         // Apply dynamic range boost from Radar Dish (if present)
73         boolean hasRadar = false;
74         for (Instance inst : instances) {
75             if (inst instanceof RadarDish && inst.isAlive()) {
76                 hasRadar = true;
77                 break;
78             }
79         }
80         double rangeBoost = hasRadar ? 1.25 : 1.0;
81         for (Instance inst : instances) {
82             if (inst instanceof Building) {
83                 inst.rangeMult = rangeBoost;
84             }
85         }
86
87         // Update all instances
88         for (Instance instance : instances) {
89             if (instance.isAlive()) {
90                 instance.update(deltaTime);
91                 instance.routine(deltaTime);
92                 for (Turret turret : instance.turrets) {
93                     turret.update(deltaTime);
94                 }
95                 for (Weapon weapon : instance.weapons) {
96                     weapon.update(deltaTime);
97                 }
98                 for (Utilities.Animation anim : instance.anims) {
99                     anim.update(deltaTime);
100                 }
101                 // Remove dead animations
102                 instance.anims.removeIf(anim -> !anim.isAlive());
103             }
104         }
105
106         // Add queued instances and remove dead ones
107         if (!iQueue.isEmpty()) {
108             instances.addAll(iQueue);
109             iQueue.clear();
110         }
111         instances.removeIf(instance -> !instance.isAlive());
112     }
113
114     /**
115      * Determine which cursor should be displayed based on game mode
116      */
117     public String getDesiredCursor(boolean buildMode, boolean repairMode, boolean sellMode) {
118         if (buildMode) {
119             return "normal_cursor"; // placeholder for build cursor
120         } else if (repairMode) {
121             return "repair_cursor";
122         } else if (sellMode) {
123             return "sell_cursor";
124         }
125         return "normal_cursor";
126     }
127 }
128

```


update() ของคลาส Turret จะแยกแยะว่า parent มันคืออะไรและออกอาการต่างกันไป

Buildings

- “เดา” ว่าศัตรูจะอยู่ตรงไหนโดยคำนวณจากความเร็วของกระสุน ความเร็วของศัตรู และระยะห่าง
- หมุนป้อมโดยใช้ deltaTime เพื่อความสมจริง

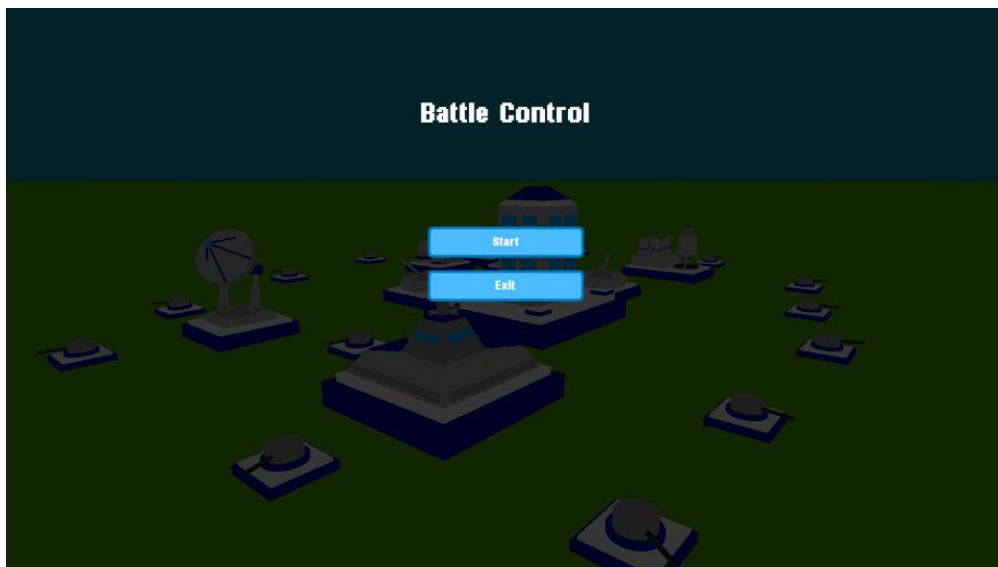
Enemies

- ใช้ระบบน้ำหนักและสุ่มในการตัดสินใจว่าจะเล็งอะไร
- เคลื่อนที่ช้าลงเมื่อมีเป้าหมาย
- หมุนป้อมโดยใช้ deltaTime เพื่อความสมจริง

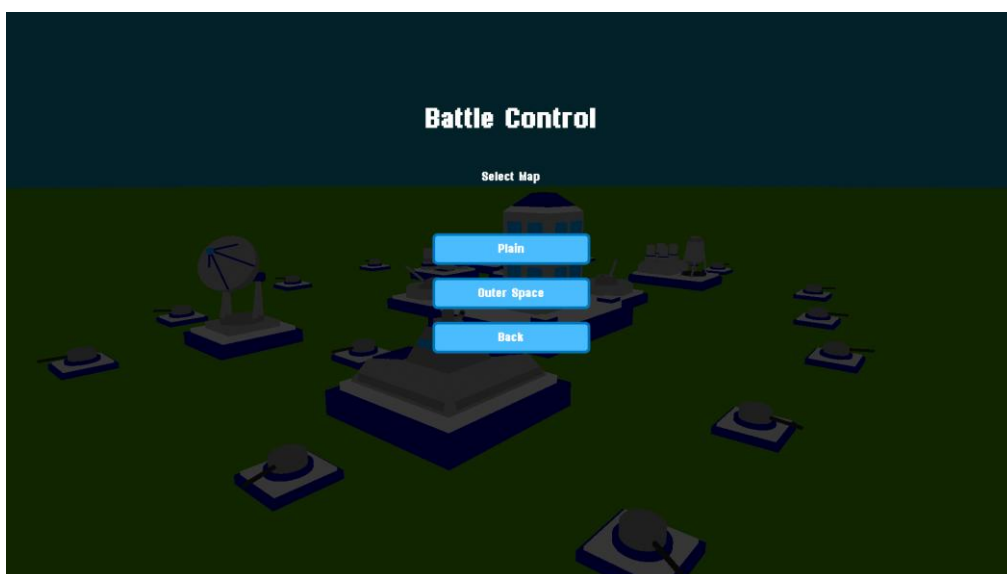
```
Gamma GammaJava
1617 void update(float deltaTime) {
1618     if (GameManager.getInstance().getPower() < 0 && parent instanceof Building) {
1619         deltaTime = deltaTime / 2;
1620     }
1621     if (parent instanceof Enemy) {
1622         // enemy targeting logic - target building cells
1623         Enemy eParent = (Enemy) parent;
1624         intervalTimer += deltaTime;
1625         cooldownTimer += deltaTime;
1626
1627         if (isTargeting) {
1628             // decelerate while targeting
1629             if (eParent.speed > eParent.minSpeed)
1630                 eParent.speed -= eParent.dec * deltaTime;
1631
1632             if (durationTimer >= maxDuration() || bTarget == null) {
1633                 isTargeting = false;
1634                 bTarget = null;
1635                 durationTimer = 0;
1636                 cooldownTimer = 0; // Start cooldown after targeting ends
1637             } else {
1638                 double targetX = bTarget.x * location.cellSize + location.cellSize / 2.0;
1639                 double targetY = bTarget.y * location.cellSize + location.cellSize / 2.0;
1640
1641                 double dx = targetX - exactX;
1642                 double dy = targetY - exactY;
1643                 double targetAngle = Math.atan2(dy, dx);
1644
1645                 double angleDiff = targetAngle - facing;
1646                 while (angleDiff > Math.PI)
1647                     angleDiff -= 2 * Math.PI;
1648                 while (angleDiff < -Math.PI)
1649                     angleDiff += 2 * Math.PI;
1650
1651                 if (Math.abs(angleDiff) > 0.001) {
1652                     double maxRotation = Math.toRadians(rot * parent.speedMult) * deltaTime;
1653                     if (Math.abs(angleDiff) <= maxRotation) {
1654                         facing = targetAngle;
1655                     } else {
1656                         facing += Math.signum(angleDiff) * maxRotation;
1657                     }
1658                 }
1659
1660                 // If close enough like 5 degrees then durationTimer += deltaTime; and fire()
1661                 // the weapons
1662                 if (Math.abs(angleDiff) < Math.toRadians(5)) {
1663                     durationTimer += deltaTime;
1664                     fire(facing, null);
1665                 }
1666             }
1667         } else {
1668             // Check for new targets
1669             if (intervalTimer >= interval && cooldownTimer >= cooldown) {
1670                 intervalTimer = 0;
1671
1672                 if (Math.random() < chance) {
1673                     // try to find a building cell in range
1674                     Point target = findBuilding();
1675                     if (target != null) {
1676                         bTarget = target;
1677                         isTargeting = true;
1678                     }
1679                 }
1680             }
1681
1682             // rotate to parent.facing when no target
1683             double angleDiff = parent.facing - facing;
1684             while (angleDiff > Math.PI)
1685                 angleDiff -= 2 * Math.PI;
1686             while (angleDiff < -Math.PI)
1687                 angleDiff += 2 * Math.PI;
1688
1689             if (Math.abs(angleDiff) > 0.001) {
1690                 double maxRotation = Math.toRadians(rot * parent.speedMult) * deltaTime;
1691                 if (Math.abs(angleDiff) <= maxRotation) {
1692                     facing = parent.facing;
1693                 } else {
1694                     facing += Math.signum(angleDiff) * maxRotation;
1695                 }
1696             }
1697
1698             recover();
1699             // accelerate back to normal speed
1700             ((Enemy) parent).speed += ((Enemy) parent).acc * deltaTime;
1701             if (((Enemy) parent).speed > ((Enemy) parent).maxSpeed) {
1702                 ((Enemy) parent).speed = ((Enemy) parent).maxSpeed;
1703             }
1704         }
1705     } else if (parent instanceof Building) {
1706         if (isTargeting) {
1707             if (eTarget == null || !eTarget.isAlive() || Math.sqrt(Math.pow(eTarget.exactX - exactX, 2)
1708                 + Math.pow(eTarget.exactY - exactY, 2)) > range * parent.rangeMult * location.cellSize) {
1709                 isTargeting = false;
1710                 eTarget = null;
1711             } else {
1712                 // Predict enemy position
1713                 Point pos = predict(eTarget);
1714                 if (pos != null) {
1715                     double targetX = pos.x;
1716                     double targetY = pos.y;
1717
1718                     double dx = targetX - exactX;
1719                     double dy = targetY - exactY;
1720                     double targetAngle = Math.atan2(dy, dx);
1721
1722                     double angleDiff = targetAngle - facing;
1723                     while (angleDiff > Math.PI)
1724                         angleDiff -= 2 * Math.PI;
1725                     while (angleDiff < -Math.PI)
1726                         angleDiff += 2 * Math.PI;
1727
1728                     if (Math.abs(angleDiff) > 0.001) {
1729                         double maxRotation = Math.toRadians(rot * deltaTime;
1730                         if (Math.abs(angleDiff) <= maxRotation) {
1731                             facing = targetAngle;
1732                         } else {
1733                             facing += Math.signum(angleDiff) * maxRotation;
1734                         }
1735                     }
1736
1737                     // If close enough like 1 degree then fire() the weapons, lighter because
1738                     // building is stationary
1739                     if (Math.abs(angleDiff) < Math.toRadians(1)) {
1740                         fire(facing, eTarget);
1741                     }
1742                 }
1743             }
1744         } else {
1745             Enemy target = findEnemy();
1746             if (target != null) {
1747                 eTarget = target;
1748                 isTargeting = true;
1749             }
1750         }
1751     }
1752 }
1753 }
1754 }
```

[illegible]

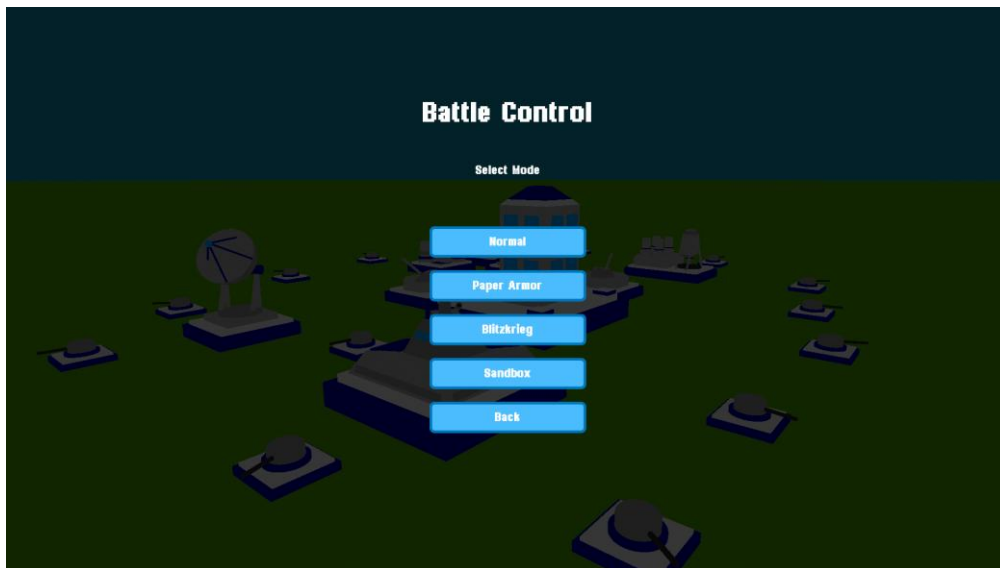
GUI



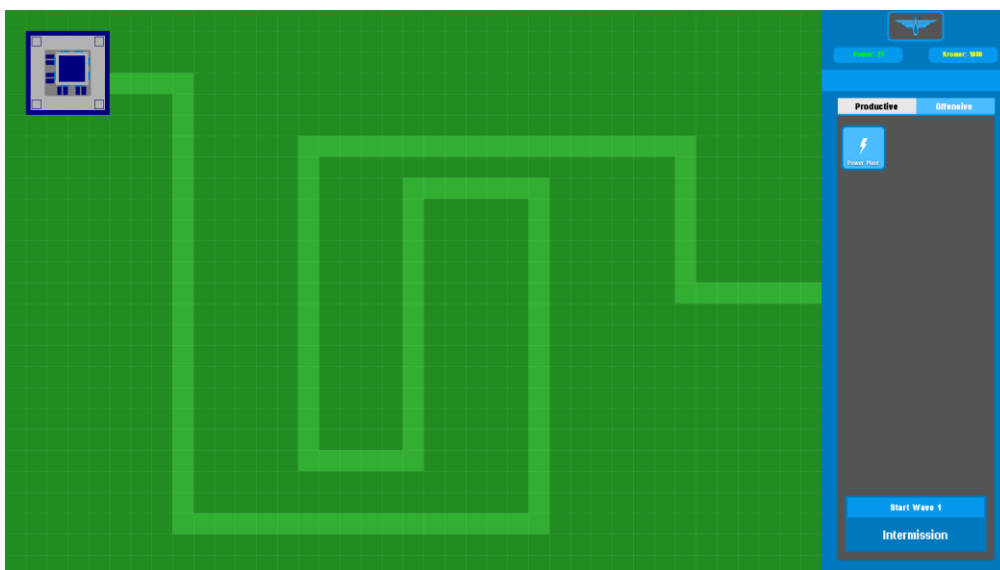
หน้าหลัก กดเล่นหรือกดออก



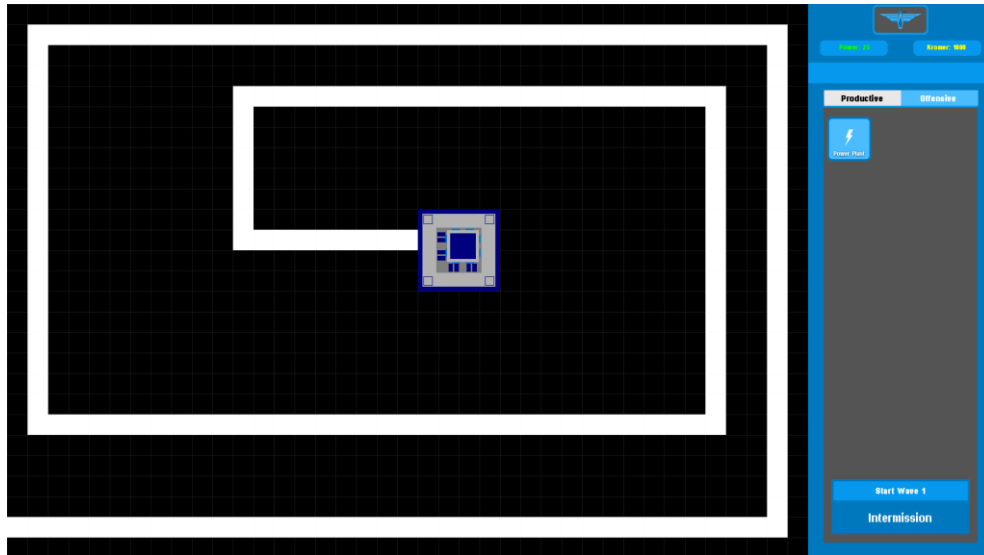
หน้าเลือกแมพ เลือกแมพหรือกดกลับไปหน้าหลัก



หน้าเลือกโหมด เลือกโหมดหรือกดกลับไปเลือกแมพ



หน้าเกมด่าน Plain เล่นหรือกด ESC เพื่อหยุดแล้วกดต่อ ออกไปหน้าหลัก หรือออกจากเกม



ด่าน Outer Space

Event Handling

InputManager มี method ที่ต้องประกาศจาก KeyEvent และ MouseEvent ทั้งหมด ส่วน logic อาจจะ
ไปอยู่ใน manager อื่น

```

Gamma - InputManager.java

1 import java.awt.event.KeyEvent;
2 import java.awt.event.MouseEvent;
3 import javax.swing.SwingUtilities;
4
5 /**
6  * Centralized input handler for the game.
7  * Manages all keyboard and mouse input events.
8  *
9  * Responsibilities:
10  * - Key press/release handling (pause, repair, sell, show hitboxes)
11  * - Mouse press/release handling (UI interactions, build mode, etc.)
12  * - Mouse movement tracking (position updates)
13  * - Tab selection (productive/offensive)
14  * - Modal state transitions (pause/unpause, build/repair/sell modes)
15  */
16 public class InputManager {
17
18     // Reference to Gamma for state access and modification
19     private Gamma gamma;
20
21     public InputManager(Gamma gamma) {
22         this.gamma = gamma;
23     }
24
25     /**
26      * Handle key released events (primary input trigger point)
27      */
28     public void handleKeyReleased(KeyEvent e) {
29         // Ignore non-tic keys when paused or game over
30         if (gamma.currentState == Gamma.GameState.PAUSED || gamma.currentState == Gamma.GameState.GAME_OVER)
31             return;
32         if (e.getKeyCode() != KeyEvent.VK_ESCAPE) {
33             return;
34         }
35
36         switch (e.getKeyCode()) {
37             case KeyEvent.VK_M:
38                 gamma.showHitboxes = !gamma.showHitboxes;
39                 break;
40
41             case KeyEvent.VK_ESCAPE:
42                 if (gamma.currentState == Gamma.GameState.IN_GAME) {
43                     gamma.currentState = Gamma.GameState.PAUSED;
44                     gamma.gameRunning = false; // freeze update
45                 } else if (gamma.currentState == Gamma.GameState.PAUSED) {
46                     gamma.currentState = Gamma.GameState.IN_GAME;
47                     gamma.gameRunning = true; // resume update
48                     gamma.lastUpdateTime = System.currentTimeMillis();
49                 }
50                 // GAME_OVER state: ESC does nothing, must click Menu button
51                 break;
52
53             case KeyEvent.VK_F:
54                 gamma.getBuildingManager().toggleRepairMode();
55                 break;
56
57             case KeyEvent.VK_K:
58                 gamma.getBuildingManager().toggleSellMode();
59                 break;
60         }
61     }
62
63     /**
64      * Handle mouse press events
65      */
66     public void handleMousePressed(MouseEvent e) {
67         // Menu (left menu, map select, mode select) - handled in paintComponent
68         if (gamma.currentState != Gamma.GameState.IN_GAME && gamma.currentState != Gamma.GameState.PAUSED) {
69             if (SwingUtilities.isLeftMouseButton(e)) {
70                 gamma.m1 = true;
71                 gamma.s1Hold = true;
72             } else if (SwingUtilities.isRightMouseButton(e)) {
73                 gamma.m2 = true;
74                 gamma.s2Hold = true;
75             }
76             return;
77         }
78
79         // Paused state: set flags but let paintComponent handle logic
80         if (gamma.currentState == Gamma.GameState.PAUSED) {
81             if (SwingUtilities.isLeftMouseButton(e)) {
82                 gamma.m1 = true;
83                 gamma.s1Hold = true;
84             } else if (SwingUtilities.isRightMouseButton(e)) {
85                 gamma.m2 = true;
86                 gamma.s2Hold = true;
87             }
88             return;
89         }
90
91         // In-game UI interactions - tab selection
92         if (gamma.mx >= 150 && gamma.mx <= gamma.umid && gamma.my >= 160 && gamma.my <= 200) {
93             gamma.tabSelected = "productive";
94         } else if (gamma.mx >= gamma.umid && gamma.mx <= gamma.umid + 130 && gamma.my >= 160 && gamma.my <= 200) {
95             gamma.tabSelected = "offensive";
96         }
97
98         // Left mouse button
99         if (SwingUtilities.isLeftMouseButton(e)) {
100             gamma.m1 = true;
101             gamma.s1Hold = true;
102         }
103
104         // Right mouse button
105         if (SwingUtilities.isRightMouseButton(e)) {
106             gamma.m2 = true;
107             gamma.s2Hold = true;
108         }
109         // Clear all build/repair/sell modes on right click
110         gamma.getBuildingManager().clearAllModes();
111     }
112
113     /**
114      * Handle mouse release events
115      */
116     public void handleMouseReleased(MouseEvent e) {
117         if (SwingUtilities.isLeftMouseButton(e)) {
118             gamma.m1 = false;
119             gamma.s1Hold = false;
120         } else if (SwingUtilities.isRightMouseButton(e)) {
121             gamma.m2 = false;
122             gamma.s2Hold = false;
123         }
124     }
125
126     /**
127      * Handle mouse dragged events (update position)
128      */
129     public void handleMouseDragged(MouseEvent e) {
130         updateMousePosition(e);
131     }
132
133     /**
134      * Handle mouse moved events (update position)
135      */
136     public void handleMouseMoved(MouseEvent e) {
137         updateMousePosition(e);
138     }
139
140     /**
141      * Update mouse position and determine if cursor is over game area or UI
142      */
143     private void updateMousePosition(MouseEvent e) {
144         gamma.mx = e.getX();
145         gamma.my = e.getY();
146
147         if (gamma.mx >= 0 && gamma.mx < Gamma.GAME_WIDTH && gamma.my >= 0 && gamma.my < Gamma.HEIGHT) {
148             gamma.status = "game";
149         } else {
150             gamma.status = "ui";
151         }
152     }
153
154     /**
155      * Empty stubs for unused event methods (required by listeners)
156      */
157     public void handleKeyPressed(KeyEvent e) {
158         // Not used
159     }
160
161     public void handleKeyTyped(KeyEvent e) {
162         // Not used
163     }
164
165     public void handleMouseClicked(MouseEvent e) {
166         // Not used
167     }
168
169     public void handleMouseEntered(MouseEvent e) {
170         // Not used
171     }
172
173     public void handleMouseExited(MouseEvent e) {
174         // Not used
175     }
176
177 }

```

บทที่ 3

สรุปผลและข้อเสนอแนะ

ปัญหาที่พบ

- ความไม่มีความรู้และความสามารถของผู้ทำ ทำให้การดำเนินการโดยรวมแย่
- พอเริ่มทำไปเยอะ ๆ แล้วมันเริ่มหาปัญหายาก ต่อให้เจอก็แก้ยาก
- ต้องไล่ทำใหม่ไปรอบนึงเพราะโค้ดเลอะมาก
- เมื่อมี loop เยอะ ๆ ทำให้ต้องเกิดการ optimization แต่ต้อง refactor codebase ทั้งหมด เหมือนต้องทำใหม่ทั้งโปรเจค
- เวลาที่ใช้ทำน้อย ปล่าวหรอก ผมผลิตวันประกันพุงเองแหละ 🙄

จุดเด่นของโปรแกรม

- แนวคิดเกมแบบ RTS + Tower Defense
- Assets ทำเอง
- Algorithm targeting แปลก ๆ
- สามารถต่อยอดเยอะมากได้หาก codebase ดีกว่านี้

ข้อเสนอแนะ

- ปรับปรุง wiring เพื่อให้การอ่านโค้ดและเพิ่มสิ่งของง่าย
- เพิ่มด่าน สิ่งก่อสร้าง และศัตรู
- เพิ่มความหลากหลายของความสามารถแต่ละอย่าง